

Prepared in accordance with IEEE 830 / ISO/IEC/IEEE 29148 SRS guidelines

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

for

Hospital Appointment Management System

A Web-Based Platform for Booking, Managing and Tracking Medical Appointments

Document Version	1.0
Document Status	Final – For Faculty Evaluation
Prepared For	GLA University
Course / Program	B.Tech
Prepared By	Abhinav Pandey (Roll No: 22515000006)
Project Guide	Mr. Gautam Mukherjee
Date	18/08/2026

Table of Contents

1. Introduction.....	4
1.1 Purpose.....	4
1.2 Document Conventions.....	4
1.3 Intended Audience and Reading Suggestions.....	4
1.4 Project Scope.....	4
1.5 References.....	4
2. Overall Description.....	5
2.1 Product Perspective.....	5
2.2 Product Features (Summary).....	5
2.3 User Classes and Characteristics.....	5
2.4 Operating Environment.....	5
2.5 Design and Implementation Constraints.....	6
2.6 Assumptions and Dependencies.....	6
3. System Features (Functional Requirements).....	6
3.1 Module: User Registration & Authentication.....	6
3.2 Module: Patient Portal.....	7
3.3 Module: Doctor Portal.....	7
3.4 Module: Administrator Portal.....	8
3.5 Module: Notifications.....	8
4. External Interface Requirements.....	8
4.1 User Interfaces.....	8
4.2 Hardware Interfaces.....	9
4.3 Software Interfaces.....	9
4.4 Communication Interfaces.....	9
5. Use Case Summary.....	9
5.1 Primary Flow – Appointment Booking (Patient).....	10
5.2 Use Case: Doctor Manages Weekly Availability.....	10
5.3 Use Case: Doctor Records Consultation Notes.....	11
5.4 Use Case: Admin Manages Doctors & Departments.....	11
5.5 Use Case: Admin Generates Operational Reports.....	11
5.6 Use Case: Patient Cancels or Reschedules an Appointment.....	12
6. Non-Functional Requirements.....	12
6.1 Performance Requirements.....	12
6.2 Security Requirements.....	13
6.3 Usability.....	13
6.4 Reliability & Availability.....	13
6.5 Maintainability & Portability.....	13

6.6 Scalability.....	13
7. Software Testing Considerations.....	14
8. Proposed System Architecture.....	15
8.1 Suggested Technology Stack.....	15
9. Data Model Overview (for Database Planning).....	16
9.1 Core Entities.....	16
9.2 Key Relationships.....	18
10. Project Timeline & Work Breakdown.....	18
11. Requirements Traceability Matrix.....	18
12. Risk Analysis.....	20
13. Future Enhancements (Out of Current Scope).....	20
14. Glossary.....	20

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements for TripEase, a web-based Hospital Appointment Management System designed to simplify the process of booking, managing and tracking medical appointments between patients and hospital staff. The document is intended to serve as the single source of truth for the development team, project guide, and evaluation committee regarding what the system will do, who will use it, and the constraints under which it must be built. It is written to be used for requirement analysis, UI/UX design, database and API planning, software testing, and final project documentation.

1.2 Document Conventions

Functional requirements are uniquely identified using the format FR-XX, and are grouped by module. Each requirement is written using the keyword “shall” to indicate a mandatory capability of the system, consistent with IEEE 830 / ISO/IEC/IEEE 29148 conventions. Priority levels used throughout this document are High (must be implemented for a working system), Medium (important but can be simplified if time is short) and Low (optional / future enhancement).

1.3 Intended Audience and Reading Suggestions

- Students / Development Team – to understand what to build, module by module.
- Project Guide / Faculty Evaluator – to assess completeness, correctness and feasibility of scope.
- UI/UX Designer – to design screens based on Section 4 and Section 3 workflows.
- Database / Backend Developer – to design schema and APIs based on Section 8 and Section 9.
- Software Tester – to derive test cases directly from the functional requirements in Section 3.

1.4 Project Scope

TripEase will allow patients to register, search for doctors by department/specialization, view real-time doctor availability, book/cancel/reschedule appointments, and view their appointment history and prescriptions. Doctors will be able to manage their availability, view their daily/weekly appointment list, and record consultation notes after a visit. Administrators (hospital staff) will manage doctor and department records, oversee all appointments, and generate basic operational reports.

The scope is deliberately kept realistic for a college-level team project. The system will manage appointment scheduling and basic medical records — it is not a full Hospital Information System (HIS). Billing/insurance, in-patient (IPD) management, pharmacy/inventory, laboratory integration, and telemedicine (video consultation) are explicitly out of scope for the current version and are listed as future enhancements in Section 10.

1.5 References

- IEEE Std 830-1998 – IEEE Recommended Practice for Software Requirements Specifications.
- ISO/IEC/IEEE 29148:2018 – Systems and Software Engineering – Requirements Engineering.
- React.js, Node.js, Express.js and MongoDB official documentation.
- Project description document / synopsis submitted by the student team.

2. Overall Description

2.1 Product Perspective

TripEase is a new, self-contained, independent web application. It is not a modification of an existing hospital system. It follows a standard three-tier web architecture (presentation, application/API, and data layer — detailed in Section 8) so that it can later be integrated with external systems such as SMS/payment gateways as the project matures beyond the academic prototype stage.

2.2 Product Features (Summary)

- Patient registration, login and profile management.
- Doctor/department search with real-time availability display.
- Online appointment booking, rescheduling and cancellation.
- Doctor dashboard for schedule and consultation management.
- Admin dashboard for managing doctors, departments and patients.
- Automated email notifications and reminders for appointments.
- Digital prescription / consultation notes after each visit.
- Basic analytical reports (appointments per department, doctor workload, cancellation rate).

2.3 User Classes and Characteristics

User Class	Description	Technical Characteristics
Patient	General public seeking medical consultation.	Basic computer/smartphone literacy; primary end-user of the booking flow.
Doctor	Registered medical practitioner of the hospital.	Moderate technical familiarity; uses the system daily to manage schedule.
Receptionist / Front-Desk Staff (optional)	Assists walk-in patients with booking.	Trained internal staff; may book appointments on behalf of patients.
Administrator	Hospital management / super-user.	Trusted internal user with full configuration and reporting access.

2.4 Operating Environment

- Client side: Any modern web browser (Chrome, Firefox, Edge, Safari) on desktop, laptop, tablet or smartphone with an active internet connection.
- Server side: Node.js runtime environment hosted on a cloud platform (e.g. Render, Railway, AWS/Azure free tier) or a local server for demonstration.
- Database: MongoDB Atlas (cloud) or local MongoDB instance.

2.5 Design and Implementation Constraints

- Must be developed and demonstrated within a single college semester by a small student team (typically 2–4 members).
- Must primarily use free / open-source technologies to keep the project cost at zero.
- Must be responsive and usable on both desktop and mobile screen sizes.
- Payment gateway integration and SMS gateway integration are treated as optional/future add-ons due to cost and verification overhead.
- The system assumes a single hospital / single branch deployment (multi-hospital/multi-tenant support is a future enhancement).

2.6 Assumptions and Dependencies

- Users have basic familiarity with using web forms and email.
- The hospital's department and doctor list is set up by the Administrator before patients begin booking.
- Reliable internet connectivity is available at both client and server ends.
- Email delivery depends on a third-party SMTP service (e.g. Gmail SMTP / SendGrid free tier).

3. System Features (Functional Requirements)

This section lists the functional requirements of the system, grouped by module. Each requirement can be traced directly to a test case (see Section 7) and a screen (see Section 4).

3.1 Module: User Registration & Authentication

Description: Allows patients and staff to securely create accounts and log in, with role-based access.

ID	Requirement	Priority
FR-01	The system shall allow a new patient to register using name, email, phone number and password.	High
FR-02	The system shall validate email format and enforce a minimum password strength on registration.	High
FR-03	The system shall authenticate registered users (Patient, Doctor, Admin) via email and password login.	High

ID	Requirement	Priority
FR-04	The system shall issue a secure session/token (e.g. JWT) upon successful login.	High
FR-05	The system shall allow a user to reset a forgotten password via a one-time email link/OTP.	Medium
FR-06	The system shall restrict access to module features based on the logged-in user's role (Patient/Doctor/Admin).	High
FR-07	The system shall allow a logged-in user to log out, invalidating the active session.	Medium

3.2 Module: Patient Portal

Description: Core booking experience for patients.

ID	Requirement	Priority
FR-08	The system shall allow a patient to search/filter doctors by department, specialization or name.	High
FR-09	The system shall display each doctor's profile including qualification, department and available time slots.	High
FR-10	The system shall allow a patient to book an appointment by selecting an available date and time slot.	High
FR-11	The system shall prevent two patients from booking the same doctor slot at the same time (no double-booking).	High
FR-12	The system shall allow a patient to view a list of their upcoming and past appointments.	High
FR-13	The system shall allow a patient to cancel an upcoming appointment.	High
FR-14	The system shall allow a patient to reschedule an upcoming appointment to another available slot.	Medium
FR-15	The system shall allow a patient to view and update their own profile and basic medical history.	Medium
FR-16	The system shall allow a patient to view/download the consultation notes or prescription issued by the doctor.	Medium

3.3 Module: Doctor Portal

Description: Enables doctors to manage their schedule and consultations.

ID	Requirement	Priority
FR-17	The system shall allow a doctor to define and update their weekly availability (days and time slots).	High
FR-18	The system shall allow a doctor to mark specific dates as unavailable (leave).	Medium

ID	Requirement	Priority
FR-19	The system shall display a doctor's confirmed appointments for a selected day/week.	High
FR-20	The system shall allow a doctor to view a patient's basic profile and past visit history before consultation.	Medium
FR-21	The system shall allow a doctor to mark an appointment as 'Completed' and add consultation notes/prescription.	High
FR-22	The system shall allow a doctor to cancel an appointment with a reason, triggering a patient notification.	Medium

3.4 Module: Administrator Portal

Description: System configuration, oversight and reporting.

ID	Requirement	Priority
FR-23	The system shall allow the Admin to add, edit, deactivate or remove doctor accounts.	High
FR-24	The system shall allow the Admin to create and manage hospital departments/specializations.	High
FR-25	The system shall allow the Admin to view and manage all registered patient accounts.	Medium
FR-26	The system shall allow the Admin to view all appointments across the hospital, with filters by date, department or doctor.	High
FR-27	The system shall allow the Admin to generate reports on appointment volume, doctor workload and cancellation rate.	Medium
FR-28	The system shall allow the Admin to post system-wide announcements/notices visible to patients.	Low

3.5 Module: Notifications

ID	Requirement	Priority
FR-29	The system shall send an email confirmation to the patient immediately after a successful booking.	High
FR-30	The system shall send an email notification when an appointment is cancelled or rescheduled by either party.	Medium
FR-31	The system shall send an automated reminder email prior to the scheduled appointment (e.g. 24 hours before).	Low

4. External Interface Requirements

4.1 User Interfaces

- Landing / Home Page – TripEase introduction, department highlights, login/register call-to-action.
- Patient Dashboard – search doctors, book appointment, view/cancel/reschedule appointments, profile.
- Doctor Dashboard – today's/weekly appointment list, availability manager, consultation notes form.
- Admin Dashboard – manage doctors/departments/patients, appointment overview, reports and charts.
- All screens shall follow a consistent, clean, responsive layout (see Section 6.3) using a calm, green-accented, hospital-appropriate colour palette suitable for a healthcare product.

4.2 Hardware Interfaces

No specialized hardware is required. The system runs on standard client devices (desktop/laptop/tablet/smartphone) with a working internet connection, and on any commodity server/cloud instance capable of running Node.js and MongoDB.

4.3 Software Interfaces

Component	Technology
Frontend	React.js (with React Router, Axios, and a component library such as Bootstrap/Tailwind/MUI)
Backend / API	Node.js with Express.js – RESTful JSON APIs
Database	MongoDB (Mongoose ODM) – NoSQL document store
Authentication	JWT (JSON Web Tokens) with bcrypt password hashing
Email Service	SMTP service (e.g. Nodemailer with Gmail/SendGrid) for notifications
Hosting (suggested)	Frontend – Vercel/Netlify; Backend – Render/Railway; Database – MongoDB Atlas

4.4 Communication Interfaces

- All client-server communication shall occur over HTTPS using RESTful JSON APIs.
- Outbound email notifications shall be sent via the SMTP protocol through the configured email service.
- (Future phase) SMS notifications may be integrated via a gateway such as Twilio.

5. Use Case Summary

The table below summarizes the primary use cases per actor. Detailed step-by-step use-case descriptions (pre-conditions, main flow, alternate flow) can be expanded from this list during design if required by the evaluation format.

Actor	Use Case	Brief Description
Patient	Register / Login	Create an account or sign in to access booking features.
Patient	Search Doctor / Department	Find a suitable doctor by specialization or department.
Patient	Book Appointment	Select an available slot and confirm a booking.
Patient	View / Cancel / Reschedule Appointment	Manage existing bookings.
Patient	View Prescription	View consultation notes issued by the doctor.
Doctor	Manage Availability	Define working days/time slots and mark leave.
Doctor	View Appointments	See the day's/week's confirmed patient list.
Doctor	Record Consultation	Add notes/prescription and mark visit complete.
Admin	Manage Doctors & Departments	Add/edit/remove doctors and departments.
Admin	Manage Patients	View or deactivate patient accounts if required.
Admin	View Reports	Analyse appointment volume, workload and cancellations.

5.1 Primary Flow – Appointment Booking (Patient)

1. Patient logs in.
2. Searches doctor by department.
3. Selects doctor and views available slots.
4. Chooses a date/time slot.
5. Confirms booking.
6. System locks the slot and saves the appointment.
7. Confirmation email is sent to the patient and the slot is removed from the doctor's available list.

5.2 Use Case: Doctor Manages Weekly Availability

Use Case ID	UC-02
Actor(s)	Doctor
Description	Allows a doctor to define recurring weekly working hours and mark specific dates as unavailable.

Preconditions	Doctor is registered and logged in.
Main Flow	1. Doctor opens the Availability Manager from the dashboard. 2. Doctor selects working days and time-slot duration. 3. Doctor saves the weekly template. 4. System validates for overlapping slots and stores the schedule. 5. Updated availability becomes visible to patients immediately.
Alternate Flow	If the doctor marks a specific date as leave, all slots on that date are removed from patient search results, and any already-booked appointments on that date trigger a cancellation notification (FR-22).
Postcondition	Doctor's availability is updated and reflected in real time for patient booking (FR-17, FR-18).

5.3 Use Case: Doctor Records Consultation Notes

Use Case ID	UC-03
Actor(s)	Doctor
Description	Enables a doctor to close out a completed appointment by adding notes and a prescription.
Preconditions	An appointment exists with status 'Confirmed' and the scheduled time has arrived or passed.
Main Flow	1. Doctor opens today's appointment list. 2. Doctor selects a patient and reviews basic history. 3. Doctor enters consultation notes and prescribed medicines. 4. Doctor marks the appointment as 'Completed'. 5. System stores the note and links it to the appointment record.
Alternate Flow	If the patient did not show up, the doctor may instead mark the appointment as 'No-show' without adding a prescription.
Postcondition	Appointment status becomes 'Completed'; the note is visible to the patient (FR-16, FR-21).

5.4 Use Case: Admin Manages Doctors & Departments

Use Case ID	UC-04
Actor(s)	Administrator
Description	Allows the administrator to maintain the master list of departments and doctor accounts.
Preconditions	Administrator is logged in with valid credentials.
Main Flow	1. Admin navigates to the Doctors & Departments section. 2. Admin adds a new department or edits an existing one. 3. Admin creates a doctor account and assigns it to a department. 4. System validates uniqueness of email and required fields. 5. New doctor becomes searchable in the patient portal.
Alternate Flow	If the admin deactivates a doctor, that doctor's future slots are hidden from search but

	historical appointment records are retained for reporting.
Postcondition	Department/doctor master data is up to date across the system (FR-23, FR-24).

5.5 Use Case: Admin Generates Operational Reports

Use Case ID	UC-05
Actor(s)	Administrator
Description	Provides the administrator with a summary view of appointment volume, doctor workload and cancellations.
Preconditions	At least one appointment record exists in the system.
Main Flow	1. Admin opens the Reports section of the dashboard. 2. Admin selects a date range and optional department filter. 3. System aggregates appointment data for the selected filters. 4. Report is displayed as summary counts/charts on screen. 5. Admin may export or print the report for evaluation records.
Alternate Flow	If no appointments match the selected filters, the system shows an empty-state message instead of a blank chart.
Postcondition	Admin has an accurate operational snapshot to support decisions (FR-27).

5.6 Use Case: Patient Cancels or Reschedules an Appointment

Use Case ID	UC-06
Actor(s)	Patient
Description	Allows a patient to change plans for an existing upcoming appointment.
Preconditions	Patient is logged in and has at least one upcoming appointment.
Main Flow	1. Patient opens 'My Appointments'. 2. Patient selects an upcoming appointment. 3. Patient chooses Cancel or Reschedule. 4. For reschedule, patient selects a new available slot with the same doctor. 5. System updates the appointment and releases/reserves slots accordingly. 6. Confirmation notification is sent to the patient.
Alternate Flow	If the chosen reschedule slot is taken by another patient before confirmation, the system shows a conflict message and asks the patient to pick another slot.
Postcondition	Appointment record reflects the new status/time, and the doctor's schedule is updated (FR-13, FR-14).

6. Non-Functional Requirements

6.1 Performance Requirements

ID	Requirement	Priority
NFR-01	The system shall load any core page within 3 seconds under normal load on a standard broadband connection.	High
NFR-02	The system shall support at least 50–100 concurrent users without noticeable performance degradation (adequate for a college demo/pilot).	Medium
NFR-03	Appointment slot confirmation shall be processed and reflected within 2 seconds of submission.	Medium

6.2 Security Requirements

ID	Requirement	Priority
NFR-04	Passwords shall be stored using a one-way hashing algorithm (e.g. bcrypt); plain-text passwords shall never be stored.	High
NFR-05	All API endpoints that access personal or medical data shall require a valid authentication token.	High
NFR-06	Role-based authorization shall ensure a Patient cannot access Doctor/Admin-only endpoints.	High
NFR-07	All data in transit shall be encrypted using HTTPS/TLS.	High
NFR-08	User input shall be validated and sanitized on both client and server to prevent injection attacks.	High

6.3 Usability

- The interface shall be simple enough for a first-time patient to book an appointment in under 5 clicks/taps.
- The system shall be fully responsive across desktop, tablet and mobile screen sizes.
- Error and success messages shall be clear, specific, and shown close to the relevant form field/action.

6.4 Reliability & Availability

- The system shall target 99% uptime during the demonstration/evaluation period.
- Booked appointment data shall not be lost due to a single server restart (persisted in the database, not in memory).

6.5 Maintainability & Portability

- The codebase shall follow a modular MVC-style structure (routes/controllers/models on the backend, reusable components on the frontend) to ease future maintenance and extension by new student contributors.
- The system shall run on any OS capable of hosting Node.js and MongoDB (Windows, Linux, macOS), and shall be accessible from any modern browser without requiring a native app install.

6.6 Scalability

Although scoped for a single hospital/branch in the academic version, the data model and API design shall avoid hard-coded single-hospital assumptions where reasonably possible, so that multi-department and, eventually, multi-branch support can be added without a full redesign.

7. Software Testing Considerations

Each functional requirement in Section 3 shall map to at least one test case. The table below gives representative examples to guide the team's test plan; the full test case document can be maintained separately (e.g. in a spreadsheet) using this format.

Test ID	Req.	Test Scenario	Expected Result
TC-01	FR-10	Book an available slot	Slot booked; confirmation shown; email sent.
TC-02	FR-11	Attempt to book an already-booked slot	System rejects with a clear 'slot unavailable' message.
TC-03	FR-13	Cancel an upcoming appointment	Appointment status changes to 'Cancelled'; slot re-opens.
TC-04	FR-06	Patient attempts to access Admin-only page	Access denied (403) / redirected to login.
TC-05	FR-02	Register with a weak password	Registration blocked with validation message.
TC-06	FR-14	Reschedule an upcoming appointment	Old slot released; new slot reserved; confirmation shown.
TC-07	FR-17	Doctor sets weekly availability	Availability saved and visible to patients immediately.
TC-08	FR-21	Doctor completes consultation with notes	Appointment marked 'Completed'; note visible to patient.
TC-09	FR-23	Admin adds a new doctor	Doctor account created and searchable by patients.
TC-10	FR-26	Admin filters appointments by department	Only matching appointments are listed.
TC-11	FR-29	Patient books a slot successfully	Confirmation email received within expected

Test ID	Req.	Test Scenario	Expected Result
			time.
TC-12	NFR-04	Inspect stored password in database	Password is stored as a bcrypt hash, never plain text.
TC-13	NFR-06	Doctor attempts to access Admin-only endpoint	Request rejected with 403 Forbidden.
TC-14	NFR-01	Load Patient Dashboard on broadband connection	Page renders within 3 seconds.
TC-15	FR-05	Request password reset with registered email	OTP/reset link is emailed and successfully resets password.
TC-16	FR-11	Two patients attempt the same slot simultaneously	Only the first request succeeds; the second sees 'slot unavailable'.
TC-17	FR-30	Doctor cancels a confirmed appointment	Patient receives a cancellation notification with reason.
TC-18	NFR-08	Submit form with script/HTML in a text field	Input is sanitized; no script execution or injection occurs.

Recommended testing types: Unit testing (backend controllers/validators using a framework such as Jest/Mocha), Integration testing (API endpoints with Postman/Jest+Supertest), and Manual UI/UX testing across at least two browsers and one mobile viewport before final submission.

8. Proposed System Architecture

Given the team's familiarity and the project's realistic academic scope, a MERN stack (MongoDB, Express.js, React.js, Node.js) three-tier architecture is recommended for TripEase. This keeps the project fully JavaScript-based end-to-end, is well documented, free to host on student-friendly free tiers, and maps cleanly onto the module breakdown in Section 3.

Layer	Responsibility
Presentation Layer	React.js SPA – Patient / Doctor / Admin UI, communicating over HTTPS / REST (JSON)
Application Layer	Node.js + Express.js REST API – Auth, Booking, Scheduling, business logic via Mongoose ODM
Data Layer	MongoDB – Users, Doctors, Appointments, Prescriptions

Figure 8.1: Three-tier architecture of TripEase – Hospital Appointment Management System.

8.1 Suggested Technology Stack

Layer	Technology
Frontend	React.js, React Router, Axios, Bootstrap / Tailwind CSS
Backend	Node.js, Express.js, JWT, bcrypt, Nodemailer
Database	MongoDB with Mongoose
Version Control	Git & GitHub
Testing	Postman (API testing), Jest (unit testing)
Deployment (suggested)	Vercel/Netlify (frontend), Render/Railway (backend), MongoDB Atlas (database)

9. Data Model Overview (for Database Planning)

The following core entities and relationships are proposed as a starting point for schema design. This is intentionally kept minimal and realistic for a student project; fields can be extended during implementation.

9.1 Core Entities

User (base account)

Field	Type	Description
_id	ObjectId	Primary key, auto-generated.
name	String	Full name of the user.
email	String	Unique login identifier.
passwordHash	String	Bcrypt-hashed password; never stored in plain text.
role	Enum	patient / doctor / admin.
phone	String	Contact number.
createdAt	Date	Account creation timestamp.

Patient (extends User)

Field	Type	Description
dateOfBirth	Date	Used for age-based validations.
gender	Enum	Patient's gender.
address	String	Contact/mailling address.
medicalHistory	String	Short free-text summary of relevant history.

Doctor (extends User)

Field	Type	Description
departmentId	ObjectId (FK)	References Department._id.
qualification	String	e.g. MBBS, MD.
experienceYears	Number	Years of practice.
availability	Array	List of {day, startTime, endTime} slot templates.

Department

Field	Type	Description
_id	ObjectId	Primary key.
name	String	e.g. Cardiology, Dermatology.
description	String	Short description shown to patients.

Appointment

Field	Type	Description
_id	ObjectId	Primary key.
patientId	ObjectId (FK)	References Patient._id.
doctorId	ObjectId (FK)	References Doctor._id.
date	Date	Appointment date.
timeSlot	String	Booked time slot, e.g. 10:00-10:15.
status	Enum	Pending / Confirmed / Completed / Cancelled.
createdAt	Date	Booking timestamp.

Prescription / Consultation Note

Field	Type	Description
_id	ObjectId	Primary key.
appointmentId	ObjectId (FK)	References Appointment._id.
notes	String	Doctor's consultation notes.
medicines	Array	List of prescribed medicine names/dosages.
issuedAt	Date	Timestamp the note was finalized.

Notification

Field	Type	Description
_id	ObjectId	Primary key.
userId	ObjectId (FK)	References User._id (recipient).
message	String	Notification text.
type	Enum	booking / cancellation / reminder / system.
isRead	Boolean	Whether the user has viewed it.
createdAt	Date	Timestamp generated.

9.2 Key Relationships

- A Department has many Doctors (one-to-many).
- A Doctor has many Appointments; a Patient has many Appointments (many-to-many via Appointment).
- An Appointment has at most one Prescription / Consultation Note (one-to-one, created after the visit).
- A User can receive many Notifications (one-to-many).

10. Project Timeline & Work Breakdown

The table below gives an indicative week-by-week plan for a single-semester college team project. Actual pacing may be adjusted based on team size and evaluation milestones set by the department.

Phase	Weeks	Key Activities	Deliverable
Requirements & Design	1-2	Finalize scope, this SRS, wireframes, DB schema	Approved SRS (this document)
Core Backend Setup	3-4	Auth, User/Doctor/Patient models, REST API skeleton	Working auth + base APIs
Patient Portal	5-6	Doctor search, booking flow, appointment history (Section 3.2)	Functional patient booking flow
Doctor & Admin Portals	7-8	Availability manager, consultation notes, admin CRUD (Section 3.3, 3.4)	Functional doctor/admin dashboards
Notifications & Polish	9	Email notifications, UI polish, responsive fixes (Section 3.5, 6.3)	End-to-end notification flow
Testing	10-11	Unit, integration and manual testing against Section 7	Test report, bug fixes
Documentation & Demo Prep	12	Finalize SRS/report, prepare demo data and presentation	Final submission package

11. Requirements Traceability Matrix

This matrix maps every functional requirement to its owning module, a representative test case, and the primary screen where it is exercised, supporting full requirement-to-test coverage for evaluation.

FR ID	Module	Test Coverage	Primary Screen
FR-01	User Registration & Authentication	TC covers registration flow	Landing / Register screen
FR-02	User Registration & Authentication	TC-05	Register screen
FR-03	User Registration & Authentication	Login flow test	Login screen
FR-04	User Registration & Authentication	Session/token test	All authenticated screens
FR-05	User Registration & Authentication	TC-15	Forgot Password screen
FR-06	User Registration & Authentication	TC-04	Role-restricted routes
FR-07	User Registration & Authentication	Logout test	All authenticated screens
FR-08	Patient Portal	Doctor search test	Patient Dashboard
FR-09	Patient Portal	Doctor profile test	Doctor Profile screen
FR-10	Patient Portal	TC-01	Book Appointment screen
FR-11	Patient Portal	TC-02, TC-16	Book Appointment screen
FR-12	Patient Portal	Appointment list test	My Appointments screen
FR-13	Patient Portal	TC-03	My Appointments screen
FR-14	Patient Portal	TC-06	My Appointments screen
FR-15	Patient Portal	Profile update test	Patient Profile screen
FR-16	Patient Portal	Prescription view test	Prescription screen
FR-17	Doctor Portal	TC-07	Availability Manager
FR-18	Doctor Portal	Leave marking test	Availability Manager
FR-19	Doctor Portal	Schedule view test	Doctor Dashboard
FR-20	Doctor Portal	Patient history view test	Doctor Dashboard
FR-21	Doctor Portal	TC-08	Consultation Notes screen
FR-22	Doctor Portal	TC-17	Doctor Dashboard
FR-23	Administrator Portal	TC-09	Admin - Doctors screen
FR-24	Administrator Portal	Department CRUD test	Admin - Departments screen
FR-25	Administrator Portal	Patient management test	Admin - Patients screen

FR ID	Module	Test Coverage	Primary Screen
FR-26	Administrator Portal	TC-10	Admin - Appointments screen
FR-27	Administrator Portal	Reports test	Admin - Reports screen
FR-28	Administrator Portal	Announcement test	Admin - Announcements screen
FR-29	Notifications	TC-11	Email (external)
FR-30	Notifications	TC-17	Email (external)
FR-31	Notifications	Reminder email test	Email (external)

12. Risk Analysis

The following risks were identified for the project timeline and technical implementation, along with mitigation strategies to be reviewed periodically by the team and project guide.

Risk	Likelihood	Impact	Mitigation Strategy
Double-booking of a slot under concurrent requests	Medium	High	Use atomic DB updates / unique index on (doctorId, date, timeSlot) to prevent race conditions.
Team member unavailability close to deadline	Medium	High	Maintain clear module ownership and weekly check-ins per Section 10 timeline.
SMTP/email service rate limits or downtime	Medium	Medium	Use a free-tier provider with generous limits (e.g. SendGrid) and log failed sends for retry.
Scope creep beyond Section 1.4 boundaries	High	Medium	Treat Section 10 (Future Enhancements) as strictly out of scope until core features are stable.
Underestimated testing time before submission	Medium	High	Start testing incrementally per module (Section 7) rather than only at the end.
Free-tier hosting cold-starts affecting demo performance	Medium	Low	Warm up the deployed server shortly before any live demonstration.
Inconsistent UI across modules built by different members	Medium	Low	Agree on a shared component library and colour palette early (Section 4.1).
Loss of local work due to no version control discipline	Low	High	Enforce regular commits/pushes to a shared Git repository (Section 8.1).

13. Future Enhancements (Out of Current Scope)

- Online payment integration for consultation fees.
- SMS notifications alongside email.

- Video consultation / telemedicine module.
- Multi-hospital / multi-branch support.
- AI-based doctor recommendation or appointment-time prediction.
- Integration with wearable/health-tracking data.

14. Glossary

Term	Definition
SRS	Software Requirements Specification
API	Application Programming Interface
REST	Representational State Transfer – a web API architecture style
JWT	JSON Web Token – used for stateless authentication
MERN	MongoDB, Express.js, React.js, Node.js – a full JavaScript web stack
CRUD	Create, Read, Update, Delete – basic data operations
OTP	One-Time Password
UI/UX	User Interface / User Experience

End of Document – TripEase Hospital Appointment Management System SRS v1.0